VoxelMap++: Mergeable Voxel Mapping Method for Online LiDAR(-inertial) Odometry

Yifei Yuan, Chang Wu, *Member, IEEE*, Yuan You, Xiaotong Kong, Ying Zhang, Qiyan Li

***Abstract*—This paper presents VoxelMap++: a voxel mapping method with plane merging which can effectively improve the accuracy and efficiency of LiDAR(-inertial) based simultaneous localization and mapping (SLAM). This map is a collection of voxels that contains one plane feature with 3DOF representation and corresponding covariance estimation. Considering total map will contain a large number of coplanar features (kid planes), these kid planes' 3DOF estimation can be regarded as the measurements with covariance of a larger plane (father plane). Thus, we design a plane merging module based on union-find which can save resources and further improve the accuracy of plane fitting. This module can distinguish the kid planes in different voxels and merge these kid planes to estimate the father plane. After merging, the father plane 3DOF representation will be more accurate than the kids plane and the uncertainty will decrease significantly which can further improve the performance of LiDAR(-inertial) odometry. Experiments on challenging environments such as corridors and forests demonstrate the high accuracy and efficiency of our method compared to other state-of-the-art methods (see our attached video[1]). By the way, our implementation VoxelMap++ is open-sourced on GitHub[2] which is applicable for both non-repetitive scanning LiDARs and traditional scanning LiDAR.**

***Index Terms*—Mapping; Union-Find; Localization; SLAM**

I. Introduction

RECENTLY, with the rapid development of 3D LiDAR technology, LiDAR(-inertial) odometry has been wildly used in various applications such as autonomous vehicles [3] [4], UAVs (Unmanned Aerial Vehicles) [5] and mobile robot [7] because of the robustness and accuracy. One important part of LiDAR(-inertial) odometry is the representation of the history map, almost all state-of-art LiDAR(-inertial) odometry requires the map can provide fast and accurate registration and can increment efficiently during the moving of robots. The most commonly used mapping method in LiDAR-based simultaneous localization and mapping (SLAM) is KD-Tree point cloud map due to its ease of use, e.g. LOAM [8], Lego-LOAM [9], LIO-SAM [10], LIO-Mapping [11], LINS [12], LILI-OM [13], FAST-LIO [1]. However, in the process of point cloud registration, it is inevitable to frequently search for K nearest neighbor points to fit the plane which is a time consume task for KD-Tree. The time complexity of the nearest find in KD-Tree is O(log(N)) which is harmful to the real-time performance. Moreover, preserving the uncertainty of point clouds is almost impossible. Because each point is a random variable of 3DOF, covariance needs to be represented using a 3×3 covariance matrix. After covariance estimation, the memory usage of the entire map will expand to four times its original size, which is clearly unaffordable in practice.

Fortunately, some voxel mapping methods based on spatial hashing have been proposed by scholars in recent years, such as LIO-Livox, Faster-LIO [14], BALM [15], BALM 2.0 [16], VoxelMap [2]. The time complexity of registration is O(1), which can guarantee the real-time performance of the system. In these methods, VoxelMap has excellent accuracy and robustness due to its uncertainty estimation ability about the plane feature. However, the downside is that it uses 6DOF to represent flat surfaces, which is a waste of memory and increases the time complexity of the total system obviously. It also does not handle the relationship between different voxels, resulting in high repetition of the same plane in the VoxelMap. If coplanar voxels can be effectively distinguished, then estimate the large plane formed by these small planes in the voxels. The uncertainty of the entire map will significantly decrease, which will further improve the positioning accuracy of online LiDAR(-inertial) odometry.

To address the above challenges, this paper proposes a novel online mergeable voxel mapping method with 3DOF plane representation termed VoxelMap++. Concretely, the contributions of this paper include:

- We promote the plane fitting and covariance estimation method in Voxelmap from 6DOF to 3DOF by using least squares estimation. This improvement is from the perspective of engineering implementation, further improving the efficiency of covariance estimation and reducing memory usage, which allows VoxelMap++ to easily adapt to various resource-constrained embedded platforms.
- We propose a novel online voxel merging method by using union-find. Each coplanar feature (kid plane) in the voxel will be regarded as the measurements of a large plane (father plane). The merging module not only arising the plane fitting accuracy and decrease the uncertainty of the total map, but also reduces the memory usage of the map.
- We compared VoxelMap++ with other state-of-art algorithms in various scenarios (structured, unstructured and degenerated) which demonstrates the superiority of the algorithm in accuracy and efficiency.
- We make the VoxelMap++ adapt different kinds of LiDARs (multi-spinning LiDARs and non-conventional

Yifei Yuan, Chang Wu, Xiaotong Kong, Ying Zhang, Qiyan Li are with the Institute of Information and Communication Engineering, University of Electronic Science and Technology of China (UESTC), Chengdu 611731, China (email:yuanyf1998@126.com; changwu@uestc.edu.cn; m13086663208@163.com).

Yifei Yuan and Yuan You have the same contribution to this paper as co-first author. Chang Wu is the Corresponding author.

1 https://www.bilibili.com/video/BV16h4y1r7Cm/

2 https://github.com/uestc-icsp/VoxelMapPlus_Public

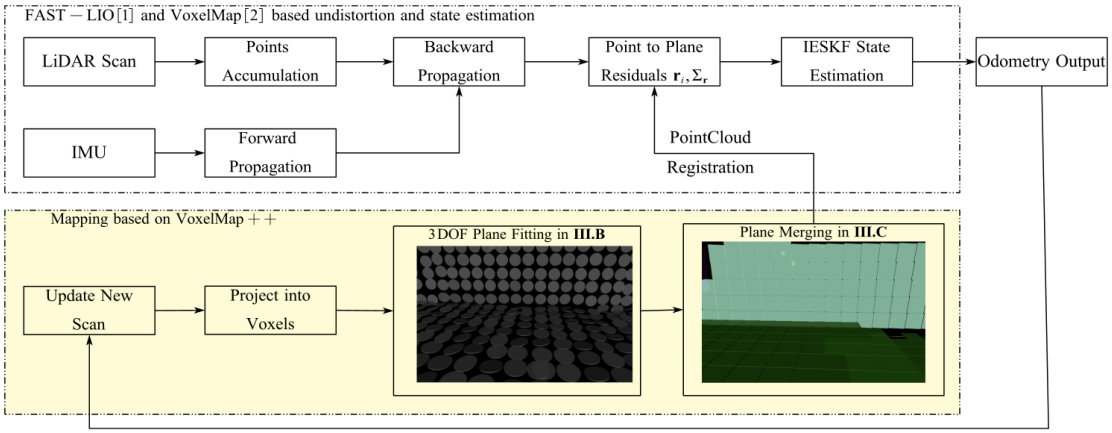


Fig. 1. System overview of VoxelMap++, the main contribution of this article is the mapping block denoted in yellow

solid-state LiDARs) and open-sourced with readability and modularity on our GitHub for sharing our findings and making contributions to the community.

II. Replated Work

LOAM [8] and its variant [9] construct the map online by incrementally registering the new scan into point clouds map based on KD-Tree, then perform voxel filtering for downsampling to reduce the number of points in the map. However, the efficiency of point cloud maps in scan registration is still low because each point-plane matching requires retrieving at least three nearest points to fit the corresponding plane. As the map increases, the cost of reconstructing KD-Tree also gradually increases. Making KD-Tree based point cloud maps unsuitable for some large-scale scenarios.

To overcome the drawback of map based on point clouds, FAST-LIO2 [17] develops an incremental kd-tree structure [18] to organize the point cloud map efficiently; MINS [19] maintains a plane patch point cloud to decrease the numbers of point in map, which contain the center point p and Hesse normal n of plane. With the efforts of these researchers, point cloud maps have gradually become practical, but this is accompanied by a series of complex map management methods.

Normal distribution transformation (NDT) [28] and its variant [19] is the most classic voxel mapping method which divided the positioning spaces into voxels. Each voxel contains a 3D Gaussian distribution by fitting the history point cloud. By calculating the points probabilistic in the voxel, state estimation is formulated as the maximum likelihood.

Recently, some state-of-art methods (e.g. puma-lio [29], Voxelmap [2] and BALM [15]) add plane assumption in the voxel, and further estimate the covariance of the plane which achieve excellent performance in robustness and efficiency. However, these methods ignore the relationship between the neighbor voxels, which leads the entire plane (such as ground) in the map will be divided into many subplanes. Moreover, due to the limited volume of each voxel, the point clouds used to fitting the plane in the voxel are limited which makes the estimation of plane representation and covariance not accurate enough.

III. Methodology

A. System Overview

The pipeline of VoxelMap++ is shown in Fig. I. The LiDAR raw points preprocessing method and state estimation method based on iterated error-state Kalman filter are similar to FAST-LIO [1]. By the way, the mapping method of this paper can be adapted to other state-of-art LiDAR(-inertial) algorithms no matter it is based on Kalman filter or optimization.

After state estimation, each point in the new scan will be projected into the corresponding voxel, then construct or update the voxel map which is organized by a Hash table (key is voxel id and value is plane fitting results $\mathcal{P}$). The voxel map construction and update is introduced in III-C2. These new points will be incrementally used for 3DOF plane fitting and covariance estimation which will be introduced in III-B. The complexity of this module will not increase with the number of points in the voxel, because all value used for fitting the plane is the form of summation which can be cached and calculated incrementally.

Then, the converging plane will be used for plane merging which will be introduced in III-C3. In this module, kid plane $\mathcal{P}^k$ in the voxels will be merged into father plane $\mathcal{P}^f$ based on union-find. Meanwhile, the plane estimation results of $\mathcal{P}^f$ will be more accurate compared with $\mathcal{P}^k$ which will obviously improve the positioning results of LiDAR(-inertial) odometry.

B. 3DOF Plane Fitting and Covariance Estimation

Like other implementations of LiDAR(-inertial) odometry [1] [2] [14] [19], we only use the plane features in the environment due to its vast availability. In each voxel, we maintain a 3DOF plane fitting results $\mathbf{n} = [a, b, d]^T$ and its covariance $\Sigma_{\mathbf{n}}$. In this subsection, we will illuminate how to fit the plane and estimate its covariance in 3DOF rather than other redundant representations.

Firstly, according to the analysis of the measurement noises for LiDAR sensors in [20], we can calculate the covariance of each LiDAR points ${}^{L}\mathbf{p}_i$ in the local LiDAR frame (1), then using pose estimation results $({}^{W}_{L}\mathbf{R}, {}^{W}_{L}\mathbf{t})$ to transform it to world frame (2). Thus, we can further estimate the LiDAR points covariance $\Sigma_{{}^{W}\mathbf{p}_i}$ in world frame (3). Detailed derivation of (1)(2)(3) can be found in [2].

$$\mathbf{A}_i = \begin{bmatrix} \boldsymbol{\omega}_i & -d_i \lfloor \boldsymbol{\omega}_i \rfloor_\times \mathbf{N}(\boldsymbol{\omega}_i) \end{bmatrix}$$
$$\Sigma_{{}^{L}\mathbf{p}_i} = \mathbf{A}_i \begin{bmatrix} \Sigma_{d_i} & \mathbf{0}_{1\times 2} \\ \mathbf{0}_{2\times 1} & \Sigma_{\boldsymbol{\omega}_i} \end{bmatrix} \mathbf{A}_i^T \tag{1}$$
$${}^{W}\mathbf{p}_i = {}^{W}_{L}\mathbf{R}\,{}^{L}\mathbf{p}_i + {}^{W}_{L}\mathbf{t} \tag{2}$$
$$\Sigma_{{}^{W}\mathbf{p}_i} = {}^{W}_{L}\mathbf{R}(\Sigma_{{}^{L}\mathbf{p}_i} + \lfloor {}^{L}\mathbf{p}_i \rfloor \Sigma_{{}^{W}_{L}\mathbf{R}} \lfloor {}^{L}\mathbf{p}_i \rfloor_\times^T) {}^{W}_{L}\mathbf{R}^T + \Sigma_{{}^{W}_{L}\mathbf{t}} \tag{3}$$

Assuming a group of coplanar LiDAR points ${}^{W}\mathbf{p}_i, (i = 1, \cdots, N)$ with covariance $\Sigma_{{}^{W}\mathbf{p}_i}$ have be determined using (3). Then, we leverage the linear least-squares method to calculate the 3DOF representation of plane [19]. Considering a plane can be extracted from this point cloud parameterized as (4) by normalizing along z-component (main-axis). This expression can be singular when the z-component of the plane normal is close to zero. While this issue can be easily resolved by calculating the projection of the point cloud to each coordinate axis, and the one with the smallest projection variance is the main-axis.

$$ax + by + z + d = 0 \tag{4}$$

Since all ${}^{W}\mathbf{p}_i$ are coplanar satisfied (4), so the least squares optimization function can be constructed as (6). After a series of identical deformations, a closed-form solution of $\mathbf{n}$ can be obtained (7).

$$\begin{bmatrix} x_1 & y_1 & 1 \\ x_2 & y_2 & 1 \\ \vdots & \vdots & \vdots \\ x_n & y_n & 1 \end{bmatrix} \mathbf{n} = \begin{bmatrix} -z_1 \\ -z_2 \\ \vdots \\ -z_n \end{bmatrix} \tag{5}$$
$$\mathbf{n} = \frac{\mathbf{A}^*}{|\mathbf{A}|}\mathbf{e} \tag{6}$$

where $\mathbf{A}^*$ is the adjugate matrix of $\mathbf{A}$, the expression of $\mathbf{A}$ and $\mathbf{e}$ is shown as

$$\mathbf{A} = \begin{bmatrix} \sum x_i x_i & \sum x_i y_i & \sum x_i \\ \sum x_i y_i & \sum y_i y_i & \sum y_i \\ \sum x_i & \sum y_i & N \end{bmatrix}$$
$$\mathbf{e} = \begin{bmatrix} -\sum x_i z_i & -\sum y_i z_i & -\sum z_i \end{bmatrix}^T \tag{7}$$

Based on the closed-form solution of $\mathbf{n}$, the uncertainty of the plane is hence

$$\boldsymbol{\Sigma}_{\mathbf{n}} = \sum_i^N \frac{\partial \mathbf{n}}{\partial {}^{W}\mathbf{p}_i} \boldsymbol{\Sigma}_{{}^{W}\mathbf{p}_i} \frac{\partial \mathbf{n}}{\partial {}^{W}\mathbf{p}_i}^T \tag{8}$$

where the covariance matrix $\boldsymbol{\Sigma}_{\mathbf{n}}$ is the uncertainty of plane 3DOF representation, which calculated from the covariance matrix of the points on the plane $\boldsymbol{\Sigma}_{{}^{W}\mathbf{p}_i}$.

Note that all elements in $\mathbf{A}$, $\mathbf{A}^*$ and $\mathbf{e}$ are the summation results, so dynamic programming [23] can be easily used to update the 3DOF plane representation incrementally in practice which will effectively reduce the complexity of fitting the plane during update introduced in III-C2.

C. Mergeable Voxel Mapping Method

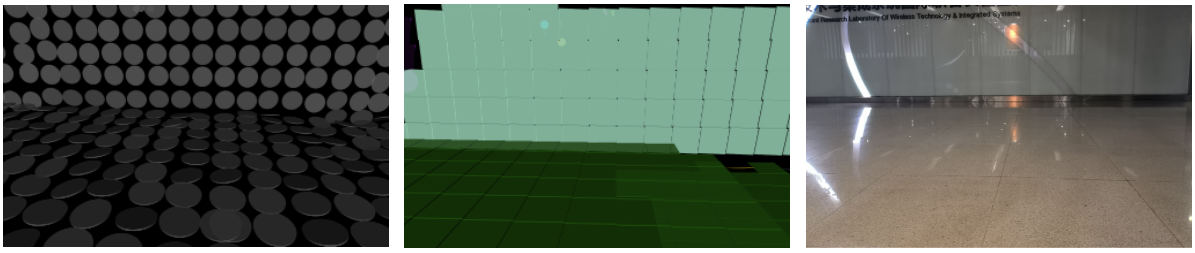
1) Motivation: In the mapping method based on the voxels, scholars always ignore the relationship between voxels no matter how the voxel maps are segmented or how small the voxels are. It is undeniable that simply treating all voxels as independent individuals is very easy for engineering practice and has strong robustness. However, if we can effectively identify the relationships between voxels (e.g. whether they belong to the same plane), then using this relationship can reasonably improve the accuracy of voxel maps and further promote the performance of LiDAR(-inertial) odometry. Meanwhile, merging the voxels with unified properties (such as coplanar) can also effectively save memory usage which is undoubtedly meaningful for commercial low-cost robots.

Based on the above ideas, we have modified the process of construction and update of voxel maps on the basis of previous research [2]. Then creatively proposed a voxel merging method based on union-find [21]. This method can distinguish the plane in different voxels termed as $\mathcal{P}^k$ and merge $\mathcal{P}^f$ to estimate the large plane composed of these small planes together, the large plane termed as $\mathcal{P}^f$. These $\mathcal{P}^k$ estimation results are regarded as the measurements with covariance of $\mathcal{P}^f$. Afterward, we estimate the plane 3DOF representation and covariance of the $\mathcal{P}^f$ using the least squares method which is significantly more accurate than the estimation results of $\mathcal{P}^k$, thereby improving the accuracy of LiDAR(-inertial) odometry.

2) Voxel Map Construction and Update: Our method is based on spatial hashing to guarantee the real-time performance of the system. Precisely, our voxel map is maintained by a hash table, the 3-dimensional space is discretized into each small voxel, the key of the hash table is the identification of each voxel while the value of the hash table is the node of union-find which represent the plane feature in this voxel.

We set the side length of each voxel to 0.5m rather than the default 3.0m in VoxelMap. It is reasonable because we replace the octo-tree with a node of union-find for plane merging which will be introduced in III-C3. Due to the lack of the feature of voxel segmentation, if we want to achieve the same resolution, it is necessary to reduce the side length of voxel. This mainly affects the memory usage of the hash table. However, by merging $\mathcal{P}^k$ in different voxels, we can free up the vast majority of voxels to reduce memory usage. In experiments shown in IV-C, we found that the memory usage of VoxelMap++ is even lower than that of VoxelMap since we do not need to maintain the planar representation in each voxel.

After the arrival of the first frame of LiDAR points, we check all of the voxels that contain enough points, then we use PCA [30] to judge whether the points in each voxel can form a plane. After that, we calculate and store the plane parameters $\mathbf{n} = [a, b, d]^T$ and their uncertainty $\Sigma_{\mathbf{n}}$ based on (6)(8). For the LiDAR points that come after the first scan, their poses in the world frame will be obtained through state estimation. Then these points are registered into the map. If there already exists a voxel in the position of the point and this voxel has not converged, then this point will be added to it and the parameters of the voxel will be updated incrementally based



(a) planes w/o merging (b) planes w merging (c) reality

Fig. 2. An example of planes w/ and w/o merging. In (a), every single plane $\mathcal{P}_i^k$ has its own parameters. In (b), the planes with the same color belong to one $\mathcal{P}^f$ and share the plane representation. In (c), it is the reality scene of background. Compared with reality, it is obvious that the plane merging method proposed in this subsection can effectively improve the accuracy of plane estimation

on (6)(8). Otherwise, we will construct a new voxel in that position.

To avoid the growing processing time with the arrival of more lidar points, the update of each voxel will stop when there are more than 50 points in it, because the uncertainty of the parameters of the plane converges when the number of points reaches 50 [2]. When a voxel stops updating, we discard all points in the voxel to save memory and only retain the plane parameters $\mathbf{n} = [a, b, d]^T$ and their uncertainty $\Sigma_{\mathbf{n}}$. Finally, the newly converging set of planes $\mathcal{S}$ will serve as input of the plane merging algorithm in III-C3.

3) Plane Merging Based on Union-Find: After voxel map update, converged plane $\mathcal{P}^k$ will be merged with their neighborhood. The plane merging algorithm is designed based on union-find shown as Algorithm 1.

Algorithm 1 Plane Merging Based on Union-Find

Require: converging planes set $\mathcal{S}$
1: **for** $\mathcal{P}_i^k \in \mathcal{S}$ **do**
2: $\mathcal{P}_i^k.f = \mathcal{P}_i^k$
3: **end for**
4: **for** $\mathcal{P}_i^k \in \mathcal{S}$ **do**
5: **for** $\mathcal{P}^n \in \mathcal{P}_i^k.neighbor$ **do**
6: **if** $\mathcal{P}^n.f$ is not coplane with $\mathcal{P}_i^k.f$ **then**
7: **continue**
8: **end if**
9: estimate ${}^N\mathcal{P}_i^k.f$ based on (10)(11)
10: **if** $\mathcal{P}^n.f == \mathcal{P}^n$ **then**
11: $\mathcal{P}^n.f = \mathcal{P}_i^k.f$
12: **else**
13: $Node = max\_kids(\mathcal{P}^n.f, \mathcal{P}_i^k.f)$
14: $\mathcal{P}_i^k.f = \mathcal{P}^n.f = Node$
15: pruning on each kids node about $\mathcal{P}_i^k.f$
16: **end if**
17: $\mathcal{P}_i^k.f = {}^N\mathcal{P}_i^k.f$
18: free the memory resource in $\mathcal{P}_i^k$ and $\mathcal{P}^n$
19: **end for**
20: **end for**

In 1-3, the new node in union-find which is the converged plane will be initialized. Then in 4-17, these new nodes will merge with other nodes in the union-find. In 5-16, each neighbor plane of $\mathcal{P}_i^k$ has been queried based on hash key and performing the plane merging about this $\mathcal{P}_i^k$ and $\mathcal{P}^n$. In 6-8, we will calculate the similarity of $\mathcal{P}_i^k$ with $\mathcal{P}^n$ based on the Mahalanobis distance shown in (9). These two planes will be merged when the Mahalanobis distance is less than the threshold given by the 95% of the χ^2 distribution. In 9, we regard the $\mathcal{P}_i^k.f$ and $\mathcal{P}^n.f$ as the measurement with covariance of the larger plane ${}^N\mathcal{P}_i^k.f$, and estimate it covariance and 3DOF representation according (10)(11) which is a simple weighted average algorithm based on minimum the trace of $\mathbf{\Sigma}_{\mathbf{n}_{N_{\mathcal{P}_i^k}}}$. In 10-12, it is two simple cases of merging nodes based on union-find which is shown as Fig.3(a,b). In 13-15, it is another complex case about plane merging about $\mathcal{P}_i^k.f$ and $\mathcal{P}^n.f$ Fig.3(c). In 16, we perform pruning after merging to maintain the maximum depth of union-find is 2 to avoid additional time consumption which is shown in Fig.3(d). In 17, we assign the results of the larger plane ${}^N\mathcal{P}_i^k.f$ to $\mathcal{P}_i^k.f$, then $\mathcal{P}^n.f$ and $\mathcal{P}_i^k.f$ have been merged successfully. Finaly, in 18, we free the resource which is the 3DOF representation and 3×3 covariance in $\mathcal{P}_i^k$ and $\mathcal{P}^n$. After merging, hundreds of voxels will share the same plane representation in $\mathcal{P}^k.f$,which will effectively reduce the memory usage.

$$\gamma = (\mathbf{n}_{\mathcal{P}_i^k} - \mathbf{n}_{\mathcal{P}^n})(\mathbf{\Sigma}_{\mathbf{n}_{\mathcal{P}_i^k}} + \mathbf{\Sigma}_{\mathbf{n}_{\mathcal{P}^n}})^{-1}(\mathbf{n}_{\mathcal{P}_i^k} - \mathbf{n}_{\mathcal{P}^n})^T \tag{9}$$

$$\mathbf{n}_{N_{\mathcal{P}_i^k}} = \frac{trace(\mathbf{\Sigma}_{\mathbf{n}_{\mathcal{P}^n}})\mathbf{n}_{\mathcal{P}_i^k} + trace(\mathbf{\Sigma}_{\mathbf{n}_{\mathcal{P}_i^k}})\mathbf{n}_{\mathcal{P}^n}}{trace(\mathbf{\Sigma}_{\mathbf{n}_{\mathcal{P}_i^k}}) + trace(\mathbf{\Sigma}_{\mathbf{n}_{\mathcal{P}^n}})} \tag{10}$$

$$\mathbf{\Sigma}_{\mathbf{n}_{N_{\mathcal{P}_i^k}}} = \frac{trace(\mathbf{\Sigma}_{\mathbf{n}_{\mathcal{P}^n}})^2\mathbf{\Sigma}_{\mathbf{n}_{\mathcal{P}_i^k}} + trace(\mathbf{\Sigma}_{\mathbf{n}_{\mathcal{P}_i^k}})^2\mathbf{\Sigma}_{\mathbf{n}_{\mathcal{P}^n}}}{\left(trace(\mathbf{\Sigma}_{\mathbf{n}_{\mathcal{P}_i^k}}) + trace(\mathbf{\Sigma}_{\mathbf{n}_{\mathcal{P}^n}})\right)^2} \tag{11}$$

Fig.2(b) is the figure of planes without merging, and Fig.2(c) is the planes after merging, planes with the same color belong to one plane and share the fitting results $\mathbf{n}$ and covariance $\Sigma_{\mathbf{n}}$.

D. State Estimation based on IESKF

The state estimation is based on an iterated extended Kalman filter similar to FAST-LIO [1] and VoxelMap [2]. Assume that we are given a state estimation prior $\hat{\mathbf{x}}_k$ with covariance $\hat{P}_k$ based on IMU propagation. This prior will be updated with the point-to-plane distance matched to form a maximum a posteriori (MAP) estimation. Specifically, the i-th

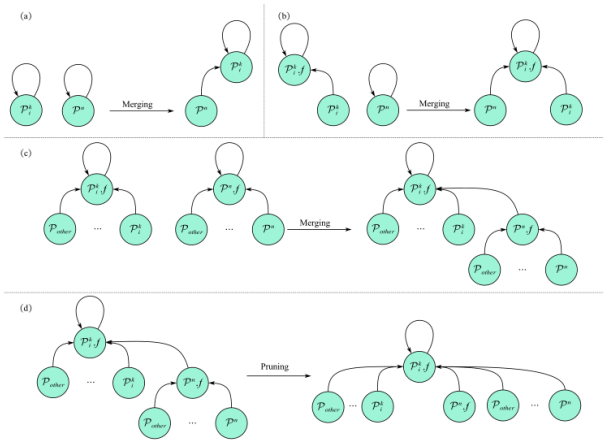


Fig. 3. The Plane Merging and Node pruning method in Algorithm 1

valid point-to-plane match leads to the observation equation shown as

$$z_i = h_i(\mathbf{x}_i, \mathbf{n}_i) = 0 \tag{12}$$
$$\approx h_i(\hat{\mathbf{x}}_i^\kappa, 0) + \mathbf{H}_i^\kappa(\mathbf{x}_i \boxminus \hat{\mathbf{x}}_i^\kappa) + \mathbf{v}_i$$
$$h_i(\hat{\mathbf{x}}_i^\kappa, 0) = \frac{\Omega^T({}_L^W\mathbf{R}{}^L\mathbf{p}_i + {}_L^W\mathbf{t}) + d}{||\Omega||} \tag{13}$$

where $h_i(\hat{\mathbf{x}}_i^\kappa, 0)$ is the point-to-plane distance in the κ iteration and Ω is the normal vector $[a, b, 1]^T$ of registration plane $\mathcal{P}_i^k.f$. $\mathcal{P}_i^k.f$ can be determined by spatial hash querying $\mathcal{P}^k$ from voxelmap firstly, then finding the root nodes of $\mathcal{P}^k$ based on union-find. The $\mathbf{v}_j \sim (0, \mathbf{R}_j)$ is the observation noise propagate from the ${}^L\mathbf{p}_i$ and $\mathbf{n}_{\mathcal{P}_i^k.f}$ which is shown as (14).

$$\Sigma_{\mathbf{n}_{\mathcal{P}_i^k.f}, {}^L\mathbf{p}_i} = \begin{bmatrix} \Sigma_{\mathbf{n}_{\mathcal{P}_i^k.f}} & \mathbf{0}_{3\times3} \\ \mathbf{0}_{3\times3} & \Sigma_{{}^L\mathbf{p}_i} \end{bmatrix}$$
$$\mathbf{R}_i = \mathbf{J}_{\mathbf{v}_i} \Sigma_{\mathbf{n}_{\mathcal{P}_i^k.f}, {}^L\mathbf{p}_i} \mathbf{J}_{\mathbf{v}_i}^T$$
$$\mathbf{J}_{\mathbf{v}_i} = [\mathbf{J}_{\mathbf{n}_i}, \mathbf{J}_{{}^L\mathbf{p}_i}], \mathbf{J}_{{}^L\mathbf{p}_i} = \frac{1}{||\Omega||}\Omega^T \times {}_L^W\mathbf{R} \tag{14}$$
$$\mathbf{J}_{\mathbf{n}_i} = \frac{1}{||\Omega||} \begin{bmatrix} x_i(1 - \frac{1}{||\Omega||^2}h_i(\mathbf{x}_i, 0)) \\ y_i(1 - \frac{1}{||\Omega||^2}h_i(\mathbf{x}_i, 0)) \\ 1 \end{bmatrix}^T$$

Finally, combining the state prior with all effective measurements, we can obtain the MAP estimation:

$$\min_{\hat{\mathbf{x}}_k^\kappa} \left\{ ||\hat{\mathbf{x}}_k^\kappa \boxminus \hat{\mathbf{x}}_k||^2_{\hat{\mathbf{P}}_k} + \sum_{i=1}^{N} ||h_i(\hat{\mathbf{x}}_k^\kappa, 0) + \mathbf{H}_i^\kappa(\mathbf{x}_k \boxminus \hat{\mathbf{x}}_k^\kappa)||_{\mathbf{R}_i} \right\} \tag{15}$$

where the first part is the state prior and the second part is the measurement observation. The detail solution is based on an iterated extended Kalman filter can refer [22].

IV. Experiments

We implemented the proposed VoxelMap++ system in C++ and Robots Operating System (ROS) on a laptop computer with 2.9GHz 8 cores and 16Gib memory. The experiment data includes open source datasets M2DGR [24] and our own challenging degenerated or unstructured datasets.

The M2DGR dataset used a Velodyne VLP-32C with $360° \times 40°$ FOV to scan the surrounding environment and obtain the 3D point cloud in 10Hz. It also used a VI-sensor Realsense d435i to obtain inertial data at 200Hz.

Our own dataset is collected via a solid-state Livox HAP LiDAR with $120° \times 25°$ FOV which is the first automotive-grade LiDAR for serial production, and a ZED 2i camera with built-in Next-Gen IMU with gyroscope, accelerator, barometer, and magnetometer. The frequency of LiDAR point cloud and IMU is 10Hz and 500Hz, respectively. These two sensors are synchronized based on IEEE 1588-2008 and the extrinsic of LiDAR and IMU ${}_L^W\mathbf{R}, {}_L^W\mathbf{t}$ have been calibrated by using LI-Init [25]. The sensors platform is shown as Fig.4. Our own datasets will also be open source to benefit researchers.

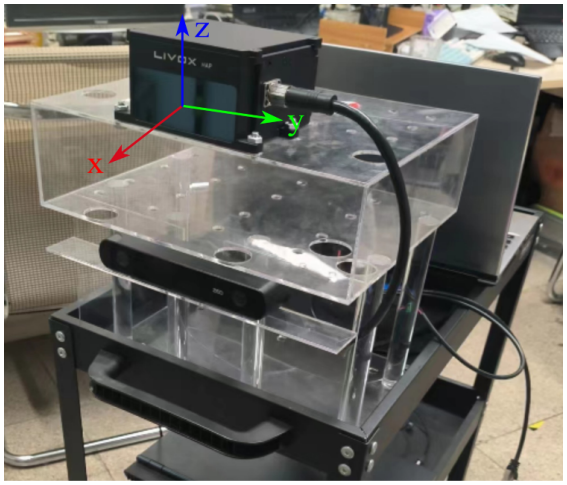


Fig. 4. Our data collection equipment has Livox HAP LiDAR and ZED 2i camera with inner IMU, these devices are well strapdown on the trolley

A. Experiment on Structured Urban

M2DGR covers the environment of structured urban, this dataset is collected by a ground robot with a full sensor-suite including an inertial measurement unit and a LiDAR, while it is also equipped with a GNSS-IMU system with real-time kinematic signals to get the dataset's ground truth. All those sensors are well-calibrated and synchronized.

Table I gives detailed information about tests route and evaluation results on A-LOAM, LeGO-LOAM, LIO-SAM, LINS, FAST-LIO2, VoxelMap and our proposed VoxelMap++. It is clear that these sequences include various environments for SLAM including long distance and short distance, indoors and outdoors, straight line and zigzag route. These scenarios are sufficient to illustrate the structured urban environment.

We use the absolute trajectory error (ATE) [26] to illuminate the accuracy, where the results for A-LOAM, LeGO-LOAM, LIO-SAM and LINS are directly drawn from [24]. ATE of FAST-LIO2, VoxelMap and VoxelMap++ is calculated based on evo [27] which is an open-source trajectory error evaluation toolkit. It is obviously that our method VoxelMap++ has better

TABLE I
ACCURACY (ATE IN METERS) COMPARISON ON M2DGR STRUCTURED URBAN SEQUENCES

Sequence	Street02(a)	Street06(b)	Street07(c)	Roomdark06(d)	Hall05(e)	Door01(f)	Lift04(g)
A-LOAM	5.299	0.628	28.940	0.314	1.065	0.274	1.323
LeGO-LOAM	20.021	1.246	35.437	0.373	1.030	0.253	1.370
LIO-SAM	4.063	0.417	28.642	0.324	1.047	0.268	X
LINS	5.636	1.742	12.009	2.205	1.010	0.258	**1.318**
FAST-LIO2	2.3236	0.4570	11.7518	0.3146	1.0227	0.2566	X
Faster-LIO	2.6667	0.4138	**11.7363**	0.3124	1.0311	0.2522	X
VoxelMap	1.7408	0.4901	13.7607	0.2944	0.9376	0.2361	X
VoxelMap++	**1.1608**	**0.4118**	12.8530	**0.2533**	**0.8991**	**0.2170**	X
Duration/s	1227	494	929	172	402	461	299
Distance/m	1484.62	479.63	1104.07	72.53	79.28	285.51	142.78
Speed/(m/s)	1.21	0.97	1.19	0.42	0.71	0.31	0.27
Description of features	day, long-term	night, straight line	night zigzag route	room, complete darkness	long-term, large overlap	outdoors to indoors	first floor to second floor by lift

accuracy than other algorithms including VoxelMap under most circumstances. It should be noted that for sequence lift04, the positioning results of VoxelMap and our method fail from a position. This is because the lift04 sequence includes rising in a closed elevator. Unfortunately, the voxel on the elevator gates has converged before closed which means the map will not change anymore on VoxelMap and VoxelMap++. When the elevator gates closed, the positioning results diverges rapidly which illuminate that VoxelMap and our proposed algorithm not applicable in dynamic scenes.

The reason our method is more accurate than VoxelMap and other state-of-art algorithm is that we adopt plane merging block which mean more points will be used to describe a single plane. In point cloud based methods such as FAST-LIO2, only 5 points are used to fit the plane commonly because of the limitation of the real-time performance of the system, which makes the estimation of the plane not accurate enough for positioning. In VoxelMap, points within a voxel are used for plane fitting, but ignore the planes relationship between voxels. Therefore, its accuracy improvement is also limited. In VoxelMap++, our plane merging block based on union-find III-C3 can better estimate the plane fitting result and covariance by making good use of the coplanar information about entire voxel map. In some cases, the laser points on the entire wall or the entire floor can be used to calculate the parameters of the single huge plane such as ground.

Fig.5 shows the LiDAR trajectories of our method and their ground truth on all sample sequences. It is obvious that the trajectories of our method are very close to the ground truth, which means our method has high accuracy. Notice that we use the same parameters in all sequences experiment rather than deliberate tuning the parameter to show better results.

B. Experiment on Challenging scenarios

In order to further test the performance of our algorithm in other challenging scenarios, such as unstructured scenes or indoors with long corridors. We design a series of experiments on these scenarios based on our own data collection platform. Unfortunately, due to the lack of hardware for GNSS-IMU systems with real-time kinematic signals. We can only use end-to-end errors, which are defined as the difference between the start point and terminal point, to compare the accuracy of different algorithms. In order to estimate the end-to-end error, we will always push the cart with the data collection platform in the path around challenging scenarios and return to the origin finally. Because the LiDAR in the platform is the non-repetitive scanning Livox HAP, so we can only use the state-of-art algorithm which is applicable for this sensor for comparison, such as FAST-LIO2, Faster-LIO, VoxelMap, and Livox's official LIO-Livox.

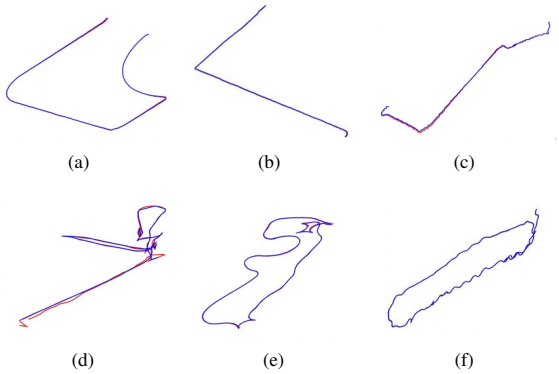


Fig. 5. (a)-(f) shows the estimated LiDAR trajectories of our method (blue) and their ground-truth (red) on all sample sequences except sequence Lift04.

1) Unstructured Forests and Grassland: Fig. 6 shows the unstructured scenarios of our experiment. we conduct experiments in a forest and grassland at the entrance of the library in UESTC.

Fig. 6. (a)(b) is the outdoor unstructured scenarios of our experiment.

As shown in Table.II, both VoxelMap and VoxelMap++ are more robust and accurate than other start-of-art methods in unstructured scenarios. This promotion is mainly due to the covariance estimation method in the point cloud $\Sigma_{w_{\mathbf{p}_i}}$, plane features $\Sigma_{\mathbf{n}}$ and observation noise $\mathbf{R}$. Our proposed VoxelMap++ is more accurate than VoxelMap because the plane merging module can make the estimation of the plane more accurate and the covariance decline in the huge plane. This means that the IESKF tends to rely on large planes with low observation covariance rather than chaotic features in forests and grasslands.

Other methods such as FAST-LIO2 regard the noise in each point-to-plane distance as a constant related to the accuracy of sensors, which means the features in the messy objects such as tree leaf and weed have the same influence as planar points. Actually, these features in messy objects do not satisfy the assumption of point in plane any more. Therefore, these features should be directly deleted or adjust their covariance scientifically. The covariance propagation in VoxelMap and VoxelMap++ is the one useful strategy.

TABLE II
END-TO-END ERROR(METERS) ON UNSTRUCTURED SEQUENCES

Sequence	loopE	loopF	loopG	loopH	loopI
LIO-Livox	5.9141	2.8160	3.6342	2.2157	1.2061
FAST-LIO2	3.2134	0.0830	4.2893	0.8175	0.1591
Faster-LIO	5.7331	1.3926	1.6632	1.7050	0.4856
VoxelMap	1.6847	0.9577	0.4433	**0.0492**	0.0894
VoxelMap++	**0.0336**	**0.0441**	**0.0389**	0.0734	**0.0406**
Route Length(m)	329.4	373.6	311.8	519.6	284.4

2) Indoor Degenerated Corridor: Fig. 7 shows the indoor degenerated scenarios of our experiment. We conduct experiments inside a building with a lot of long corridors to evaluate the performance of VoxelMap++ in indoor challenging scenarios.

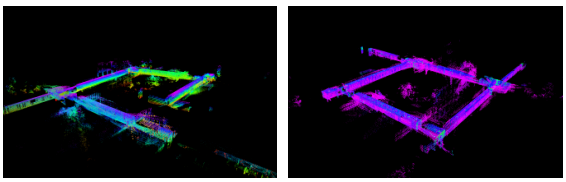
(a) (b)

Fig. 7. (a)(b) is the indoor degenerated scenarios of our experiment.

In the corridor, because of the lack of plane constraints in the forward direction which means that the measurement Jacobian will lack the normal vector like $[1, 0, 0]$. It is inevitable to cause significant cumulative errors in the x-axis of the body frame. Meanwhile, when turning at an intersection, the LiDAR will scan the next corridor without passing through, which means that the performance of the LIO attitude at this time will largely depend on the forward propagation of the IMU. This can easily lead to attitude errors, resulting in significant linear cumulative errors. In VoxelMap++, due to the advantage of plane merging, the entire floor and ceiling will be merged into two large planes, thus constraining the drift on the pitch and avoiding linear cumulative errors which is shown in Fig. 8. In fact, this phenomenon is similar to the ground segmentation in LeGO-LOAM, but it is further extended to all coplanar planes.

TABLE III
END-TO-END ERROR(METERS) ON INDOOR CORRIDOR SEQUENCES

Sequence	loop1	loop2	loop3	loop4	loop5
LIO-Livox	7.1192	18.5059	4.2932	10.0270	19.6729
FAST-LIO2	2.3812	2.8175	4.2703	13.2844	7.1070
Faster-LIO	2.3488	1.1595	8.1295	1.6455	10.1804
VoxelMap	9.3457	1.1388	3.8561	5.3355	14.2049
VoxelMap++	**0.5305**	**0.5671**	**0.2543**	**1.3094**	**1.4478**
Route Length(m)	202.9	219.7	315.2	317.1	405.0



(a) (b)

Fig. 8. The mapping result in loop1. (a) is the result of VoxelMap, (b) is the result of VoxelMap++.

As Table. III shows, other SLAM algorithms are more prone to cumulative errors in corrider. Our proposed VoxelMap++ achieves much higher accuracy than others, mainly due to the plane merging can estimate the plane representation more accurately and estimate their covariance in real-time.

C. Consumption of resources

Another advantage of our proposed VoxelMap++ is less CPU and memory resource usage compared with other state-of-art methods which is shown in Table. IV. This promotion means our method can be applied to the resource constraint embedding system such as AR equipped with iTOF and low-cost UAV.

TABLE IV
RESOURCES USAGE (AVG. COMP. TIME (MS)/ MEM USAGE (MB))

	small scale (loop1)	large scale (loopE)
FAST-LIO2	11.5985/201.86	35.4858/228.23
Faster-LIO	7.3461/135.11	15.2910/200.21
VoxelMap	5.6830/158.06	14.5815/201.22
VoxelMap++	**4.8763/126.45**	**13.8323/195.91**
Route Length(m)	202.9	329.4

From the perspective of memory usage, compared with the method based on the point cloud, we only need to maintain a hash table with save the plane information in the value which means we will not maintain millions of point clouds in the map. Compared with Voxelmap, the plane merging method in III-C3 will save the memory usage of voxels as much as possible. Ultimately, the map in VoxelMap++ will be represented as several large planes $\mathcal{P}_i^k.f$ and dozens of

planes $\mathcal{P}_j^k$ cannot be merged, rather than hundreds of voxels in VoxelMap.

From the perspective of CPU usage, compared with the method based on the point cloud, we use spatial hashing O(1) instead of KNN search O(lg(N)) based on KD-Tree. The plane parameters can be directly obtained by querying the value in the hash table instead of repeatedly performing plane fitting. Both these two advantages make the CPU usage of VoxelMap and VoxelMap++ much lower than that of pointcloud-based methods such as FAST-LIO2. Compared with VoxelMap, we perform 3DOF plane representation rather than 6DOF which means our proposed VoxelMap++ consumes less computational resources in the propagation of observation noise covariance $\mathbf{R}_i$. Meanwhile, since all inputs in the plane fitting method (6)(8) are in the form of summation. We can incrementally update the value instead of recalculating.

V. Conclusion and Limitation

This paper proposes a mergeable voxel mapping method for online LiDAR(-inertial) odometry. Compared with other methods, this method maintains the plane feature with 3DOF representation and corresponding covariance which effectively improves the calculation speed and saves memory usage. In order to improve the accuracy of plane fitting, we make full use of the relationship between voxels and then merge the coplanar voxel based on union-find after plane fitting converged. This paper also shows how to implement the proposed mapping method in an iterated extended Kalman filter-based LiDAR(-inertial) odometry. The experiment on structured open-source datasets and our own challenging datasets shows that our method can achieve better performance than other state-of-art methods.

However, our method also has some drawbacks. For example, the robustness in dynamic scenes, such as closing elevators, will drop significantly. Thus, we will consider optimizing this method from the perspective of identifying the voxel changing in the future.

Acknowledgment

This work was supported by the Science and Technology Project Fund of Sichuan Province, China, under Grant 2018sz0364. The authors would like to thank HKU-Mars-Lab because of the spirit of open-source. As far as we know, their research has widely influenced academia and industry in China.

References

[1] Xu, Wei, and Fu Zhang. "Fast-lio: A fast, robust lidar-inertial odometry package by tightly-coupled iterated kalman filter." IEEE Robotics and Automation Letters 6.2 (2021): 3317-3324.

[2] Yuan, Chongjian, et al. "Efficient and probabilistic adaptive voxel mapping for accurate online lidar odometry." IEEE Robotics and Automation Letters 7.3 (2022): 8518-8525.

[3] Wan, Guowei, et al. "Robust and precise vehicle localization based on multi-sensor fusion in diverse city scenes." 2018 IEEE international conference on robotics and automation (ICRA). IEEE, 2018.

[4] Ding, Wendong, et al. "Lidar inertial odometry aided robust lidar localization system in changing city scenes." 2020 IEEE International Conference on Robotics and Automation (ICRA). IEEE, 2020.

[5] He, Dongjiao, et al. "Point-LIO: Robust High-Bandwidth Light Detection and Ranging Inertial Odometry." Advanced Intelligent Systems (2023): 2200459.

[6] Chen, Kenny, et al. "Direct lidar odometry: Fast localization with dense point clouds." IEEE Robotics and Automation Letters 7.2 (2022): 2000-2007.

[7] Reinke, Andrzej, et al. "Locus 2.0: Robust and computationally efficient lidar odometry for real-time 3d mapping." IEEE Robotics and Automation Letters 7.4 (2022): 9043-9050.

[8] Zhang, Ji, and Sanjiv Singh. "LOAM: Lidar odometry and mapping in real-time." Robotics: Science and systems. Vol. 2. No. 9. 2014.

[9] Shan, Tixiao, and Brendan Englot. "Lego-loam: Lightweight and ground-optimized lidar odometry and mapping on variable terrain." 2018 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS). IEEE, 2018.

[10] Shan, Tixiao, et al. "Lio-sam: Tightly-coupled lidar inertial odometry via smoothing and mapping." 2020 IEEE/RSJ international conference on intelligent robots and systems (IROS). IEEE, 2020.

[11] Ye, Haoyang, Yuying Chen, and Ming Liu. "Tightly coupled 3d lidar inertial odometry and mapping." 2019 International Conference on Robotics and Automation (ICRA). IEEE, 2019.

[12] Qin, Chao, et al. "Lins: A lidar-inertial state estimator for robust and efficient navigation." 2020 IEEE international conference on robotics and automation (ICRA). IEEE, 2020.

[13] Li, Kailai, Meng Li, and Uwe D. Hanebeck. "Towards high-performance solid-state-lidar-inertial odometry and mapping." IEEE Robotics and Automation Letters 6.3 (2021): 5167-5174.

[14] Bai, Chunge, et al. "Faster-LIO: Lightweight tightly coupled LiDAR-inertial odometry using parallel sparse incremental voxels." IEEE Robotics and Automation Letters 7.2 (2022): 4861-4868.

[15] Liu, Zheng, and Fu Zhang. "Balm: Bundle adjustment for lidar mapping." IEEE Robotics and Automation Letters 6.2 (2021): 3184-3191.

[16] Liu, Zheng, Xiyuan Liu, and Fu Zhang. "Efficient and consistent bundle adjustment on Lidar point clouds." arXiv preprint arXiv:2209.08854 (2022).

[17] Xu, Wei, et al. "Fast-lio2: Fast direct lidar-inertial odometry." IEEE Transactions on Robotics 38.4 (2022). 2053-2073.

[18] Cai, Yixi et al. "ikd-Tree: An Incremental K-D Tree for Robotic Applications." ArXiv abs/2102.10808 (2021).

[19] Lee, Woosik, Yulin Yang, and Guoquan Huang. "Efficient multi-sensor aided inertial navigation with online calibration." 2021 IEEE International Conference on Robotics and Automation (ICRA). IEEE, 2021.

[20] Yuan, Chongjian, et al. "Pixel-level extrinsic self calibration of high resolution lidar and camera in targetless environments." IEEE Robotics and Automation Letters 6.4 (2021): 7517-7524.

[21] La Poutré, Johannes Antonius. New techniques for the union-find problem. Department of Computer Science, Utrecht University, 1989.

[22] D. He, W. Xu, and F. Zhang, "Embedding manifold structures into kalman filters," arXiv preprint arXiv:2102.03804, 2021.

[23] Leiserson C E, Rivest R L, Cormen T H, et al. Introduction to algorithms[M]. Cambridge, MA, USA: MIT press, 1994.

[24] Yin, Jie, et al. "M2dgr: A multi-sensor and multi-scenario slam dataset for ground robots." IEEE Robotics and Automation Letters 7.2 (2021): 2266-2273.

[25] Zhu, Fangcheng, Yunfan Ren, and Fu Zhang. "Robust real-time lidar-inertial initialization." 2022 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS). IEEE, 2022.

[26] Z. Zhang and D. Scaramuzza, "A tutorial on quantitative trajectory evaluation for visual(-inertial) odometry," in 2018 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), 2018, pp. 7244–7251.

[27] M.Grupp. "evo: Python Package for the Evaluation of Odometry and SLAM." https://github.com/MichaelGrupp/evo, 2017.

[28] Magnusson, Martin. The three-dimensional normal-distributions transform: an efficient representation for registration, surface analysis, and loop detection. Diss. Örebro universitet, 2009.

[29] Jiang, Binqian, and Shaojie Shen. "A LiDAR-inertial Odometry with Principled Uncertainty Modeling." 2022 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS). IEEE, 2022.

[30] Pearson, Karl. "LIII. On lines and planes of closest fit to systems of points in space." The London, Edinburgh, and Dublin philosophical magazine and journal of science 2.11 (1901): 559-572.